

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

**Regularization, Optimization, and Cross-Validation
on Oxford-IIIT Pet Dataset using Transfer Learning**

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Investigate the effects of weight initialization strategies, regularization techniques, batch normalization, optimizer selection, and hyperparameter tuning on MobileNetV2 transfer learning applied to fine-grained pet breed classification. Validate the best configuration using 5-fold cross-validation and report final performance on the Oxford-IIIT Pet test set.

2. Learning Outcomes

1. Evaluate the impact of weight initialization on model convergence.
2. Apply and compare regularization strategies: L2, Dropout, and Batch Normalization.
3. Compare optimizer performance: SGD, Momentum, RMSProp, and Adam.
4. Tune hyperparameters systematically (learning rate, batch size, dropout rate).
5. Compare feature extraction vs. fine tuning in transfer learning.
6. Apply 5-fold cross-validation and select the best model configuration.
7. Evaluate the final model on the test set with a full classification report.

3. Background Theory

Weight Initialization

Proper weight initialization prevents vanishing and exploding gradients at the start of training. Key schemes include:

- **Zeros:** All weights initialized to zero — causes symmetry breaking failure.
- **Random Normal:** Weights drawn from $\mathcal{N}(0, 0.01)$.
- **Xavier/Glorot:** $W \sim \mathcal{U}\left(-\sqrt{6/(n_{in} + n_{out})}, \sqrt{6/(n_{in} + n_{out})}\right)$.
- **He Normal:** $W \sim \mathcal{N}(0, \sqrt{2/n_{in}})$ — optimal for ReLU activations.

Regularization

- **L2 (Weight Decay):** $\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda \|\mathbf{w}\|_2^2$. Penalises large weights.
- **Dropout:** Randomly zeroes activations with probability p during training.
- **Batch Normalization:** Normalises layer inputs to zero mean and unit variance.

Optimizers

- **SGD:** $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$.
- **Momentum:** Accumulates gradient direction across steps.
- **RMSProp:** Adapts learning rates by dividing by a running average of gradient magnitudes.
- **Adam:** Combines Momentum and RMSProp with bias-corrected moment estimates.

5-Fold Cross-Validation

The training set is partitioned into 5 equal folds. The model is trained on 4 folds and validated on the remaining fold, rotating until each fold has served as the validation set. Final performance is reported as mean $\pm$ standard deviation across all folds.

4. Dataset

Property	Value
Dataset	Oxford-IIIT Pet
Number of Classes	37 (25 dog breeds, 12 cat breeds)
Training Images	3,680
Test Images	3,669
Image Size	Variable (resized to $224 \times 224 \times 3$)
Source	<code>tensorflow_datasets</code>

5. Model Architecture

Component	Details
Backbone	MobileNetV2 (ImageNet pretrained, 224×224 input)
Pooling	GlobalAveragePooling2D $\rightarrow$ 1280-dim vector
Optional BatchNorm	Applied after pooling in selected configurations
Dense Layer	256 units, ReLU, He Normal initialization
Optional Dropout	Rate configurable (0.0, 0.25, 0.5)
Output Layer	37 units, Softmax
Loss	Categorical Cross-Entropy
Base Optimizer	Adam, lr = 0.001
Fine Tuning Optimizer	Adam, lr = 0.0001

6. Experimental Studies

Study 1 – Weight Initialization

Four initialization strategies tested on the Dense(256) head: Zeros, Random Normal, Xavier/Glorot, He Normal. All other settings fixed (Adam lr=0.001, 5 epochs).

Initialization	Final Val Accuracy
Zeros	0.0136
Random Normal	0.9076
Xavier (Glorot)	0.9144
He Normal	0.9076

Study 2 – Regularization

Configuration	Final Val Accuracy
No Regularization	0.9076
L2 Regularization	0.9158
Dropout (0.5)	0.9103
Batch Normalization	0.9185

Study 3 – Optimizer Comparison

Optimizer	Val Accuracy	Final Loss	Time (s)
SGD	0.5530	2.2451	43.4
Momentum	0.8981	0.3714	45.3
RMSProp	0.9076	0.0364	44.4
Adam	0.9130	0.0413	47.2

Study 4 – Hyperparameter Tuning

Learning Rate:

Learning Rate	Val Accuracy
0.01	0.8560
0.001	0.9144
0.0001	0.8886
0.00001	0.2568

Batch Size:

Batch Size	Val Accuracy
16	0.9103
32	0.9130
64	0.9090

Dropout Rate:

Dropout Rate	Val Accuracy
0.00	0.8995
0.25	0.9103
0.50	0.9198

Study 5 – Feature Extraction vs. Fine Tuning

Feature extraction (frozen backbone, 10 epochs) vs. fine tuning (last 20 layers unfrozen, 10 further epochs). Final val accuracy for feature extraction: 0.9090; fine tuning (epoch 10): 0.9008.

Study 6 – 5-Fold Cross-Validation

Four configurations evaluated with 5-fold cross-validation on the full training set (224×224 , 3 epochs per fold).

Config	F1	F2	F3	F4	F5	Mean	SD
C1 Baseline	0.8927	0.8818	0.8981	0.9090	0.9117	0.8987	0.0109
C2 Dropout+BN	0.9103	0.8981	0.8927	0.9158	0.9144	0.9063	0.0092
C3 L2+BN	0.9117	0.9103	0.8913	0.9062	0.9130	0.9065	0.0079
C4 Dropout+L2+BN	0.8913	0.9117	0.9062	0.9076	0.8981	0.9030	0.0073

Best configuration: C3 L2+BN (Mean = 0.9065, SD = 0.0079).

7. Mandatory Plots

Plot 1: Training Loss vs. Epoch (Weight Initialization)

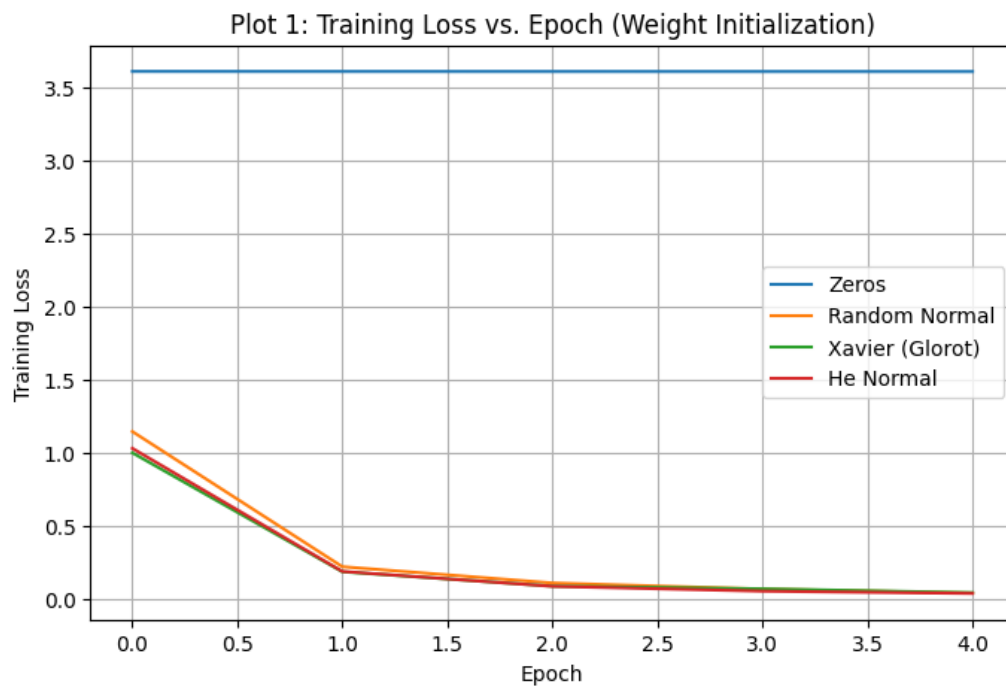


Figure 1: Training loss per epoch for four weight initialization strategies.

Inference: Zeros initialization is by far the worst with respect to accuracy. All other initializations show a decreasing loss with Xavier converging the fastest.

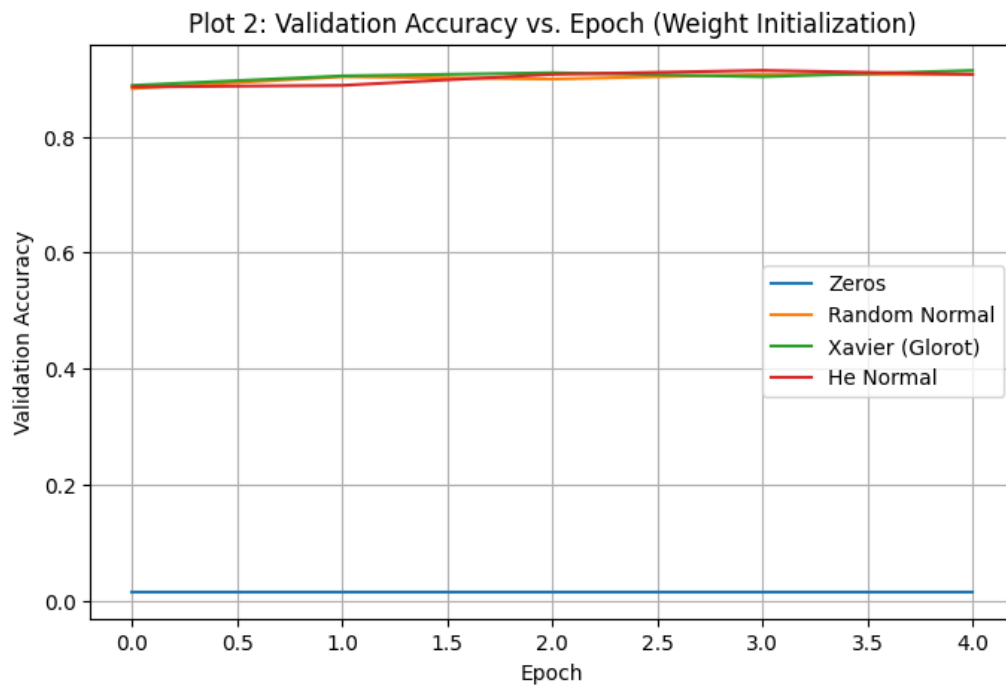
Plot 2: Validation Accuracy vs. Epoch (Weight Initialization)

Figure 2: Validation accuracy per epoch for four initialization strategies.

Inference: Zeros initialization achieves near random accuracy of 0.0136 throughout all epochs confirming the symmetry breaking problem. Xavier achieves the highest validation accuracy of 0.9144 followed closely by He Normal and Random Normal.

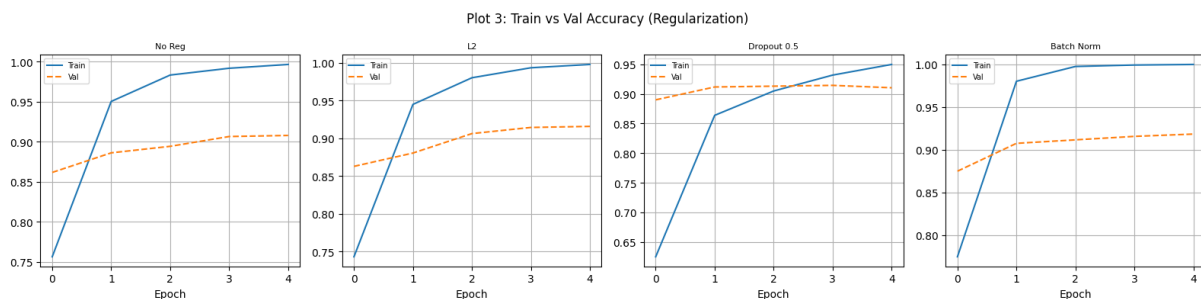
Plot 3: Train vs. Validation Accuracy (Regularization)

Figure 3: Train vs. validation accuracy for four regularization configurations.

Inference: Batch Normalization achieves the highest validation accuracy of 0.9185 across all 4 configurations, while without regularization and Dropout show a wider train-val gap indicating mild overfitting. L2 and Batch Norm both regularize effectively for this dataset size.

Plot 4: Train vs. Validation Loss (Regularization)

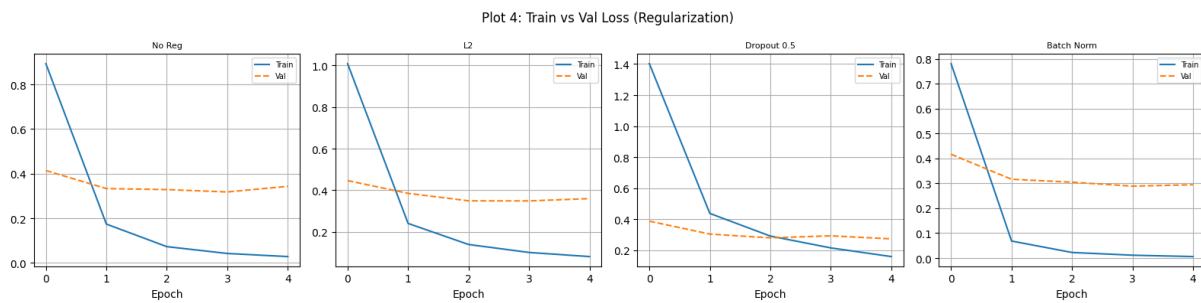


Figure 4: Train vs. validation loss for four regularization configurations.

Inference: No Regularization shows the sharpest training loss drop and the widest train-val divergence confirming overfitting. L2 and Batch Norm keep the validation loss more stable, showing they are better at controlling generalisation.

Plot 5: With vs. Without Batch Normalization

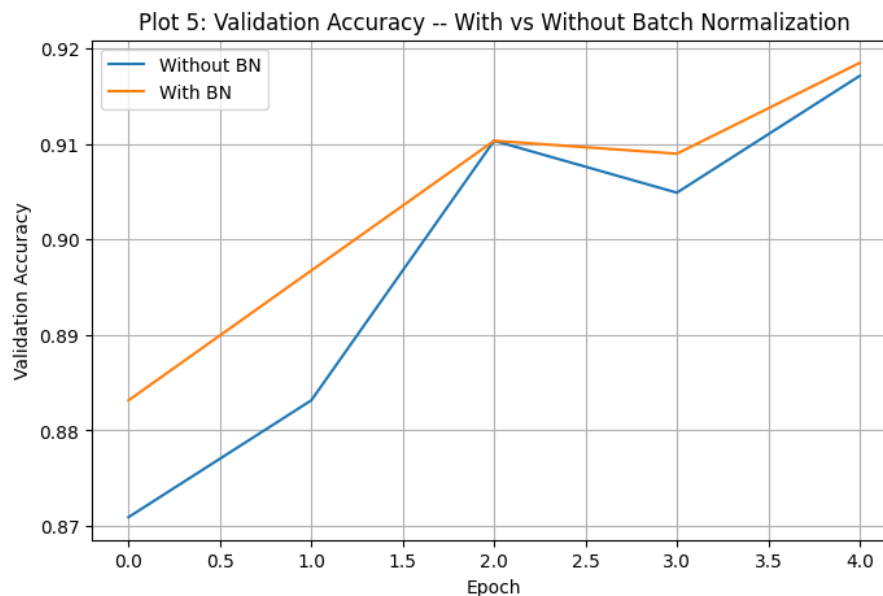


Figure 5: Validation accuracy comparison: with and without Batch Normalization.

Inference: With Batch Normalization the validation accuracy rises faster in the first 2 epochs and stabilizes at a higher value of 0.9185 compared to 0.9076 without it. This shows that normalizing the dense layer inputs speeds up convergence and improves final performance.

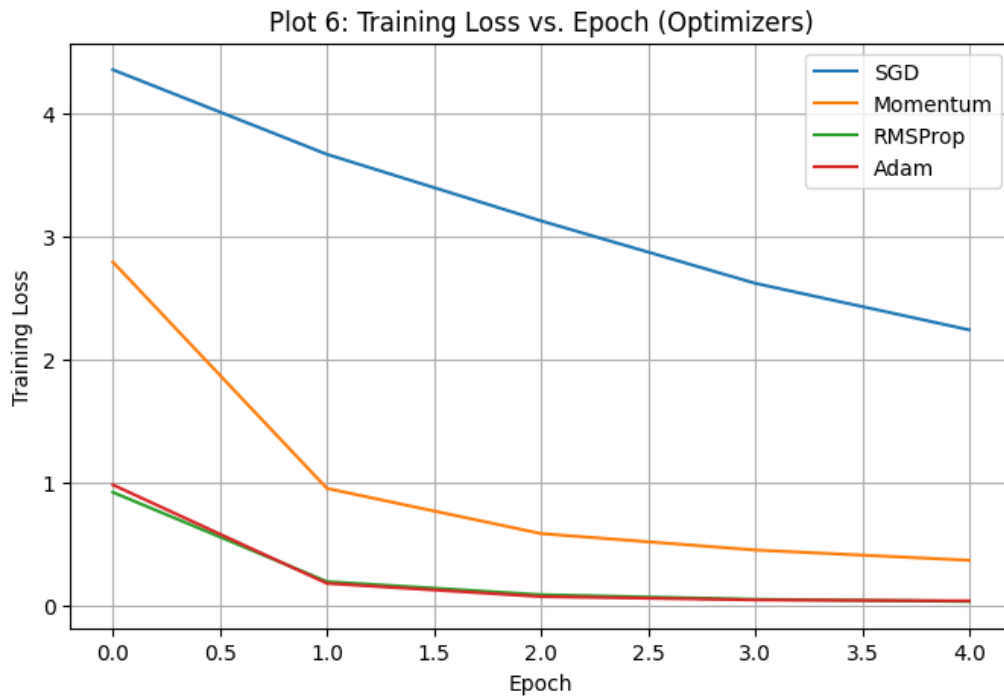
Plot 6: Training Loss vs. Epoch (Optimizers)

Figure 6: Training loss per epoch for SGD, Momentum, RMSProp and Adam.

Inference: SGD has a training loss that barely decreases over 5 epochs showing that a fixed learning rate without momentum is too slow for transfer learning. Momentum, RMSProp and Adam all decrease rapidly with RMSProp achieving the lowest final training loss.

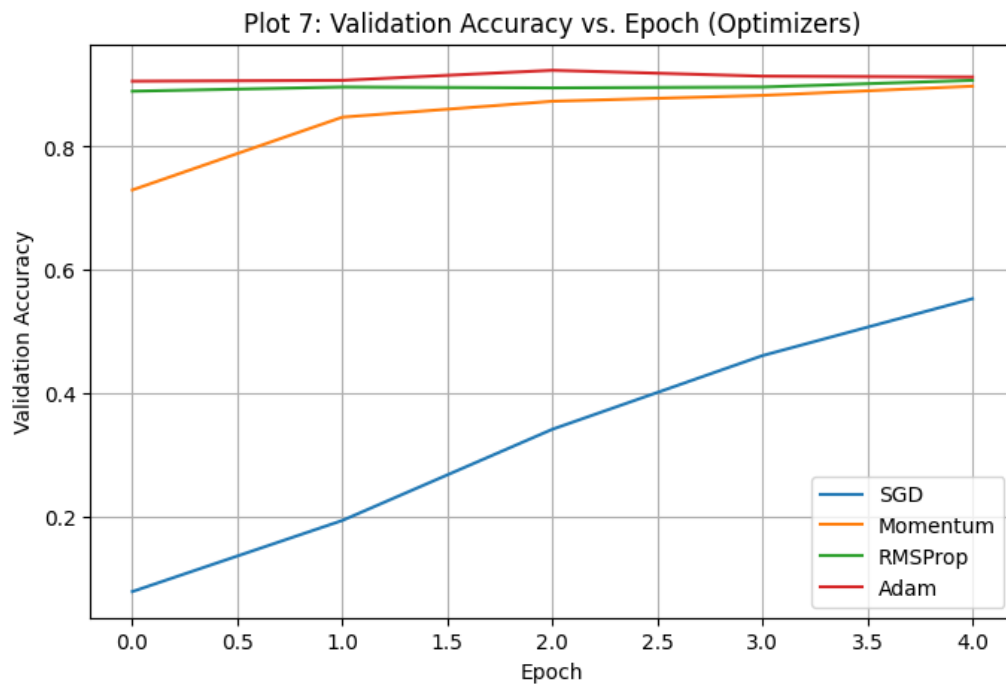
Plot 7: Validation Accuracy vs. Epoch (Optimizers)

Figure 7: Validation accuracy per epoch for SGD, Momentum, RMSProp and Adam.

Inference: Adam achieves the highest validation accuracy of 0.9130 followed by RMSProp at 0.9076 and Momentum at 0.8981. SGD only reaches 0.5530, confirming that adaptive optimizers are significantly better for fine-tuning pretrained networks.

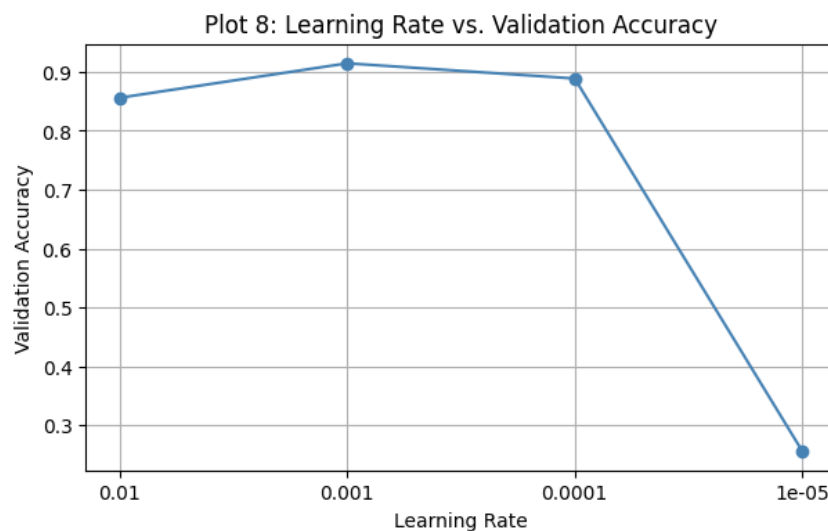
Plot 8: Learning Rate vs. Validation Accuracy

Figure 8: Final validation accuracy for learning rates 0.01, 0.001, 0.0001 and 0.00001.

Inference: Learning rate 0.001 achieves the best validation accuracy of 0.9144. At 0.01 the model overshoots the optimal weights giving 0.856 and at 0.00001 the model barely learns at

0.2568, showing that the learning rate has a sharp sweet spot around 0.001.

Plot 9: Batch Size vs. Validation Accuracy

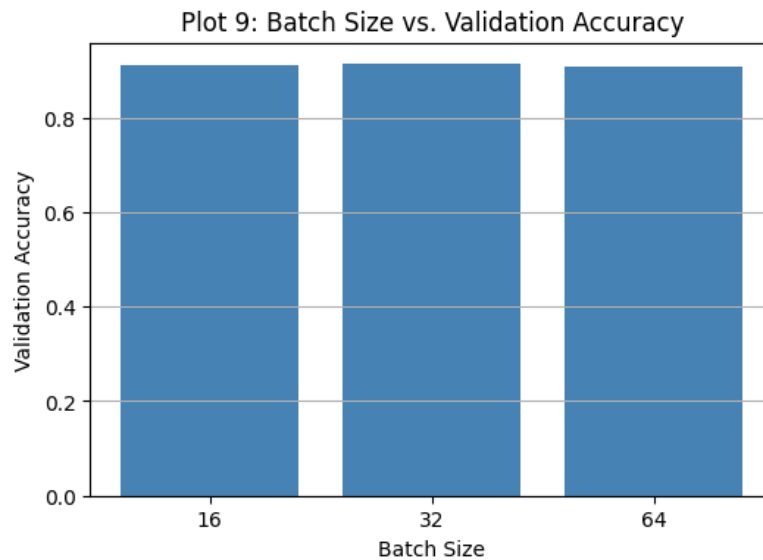


Figure 9: Final validation accuracy for batch sizes 16, 32 and 64.

Inference: Batch size 32 gives the best validation accuracy of 0.9130 with batch sizes 16 and 64 performing slightly worse. Smaller batches add noise to gradients which can help generalisation but larger batches are more stable — 32 balances both.

Plot 10: Dropout Rate vs. Validation Accuracy

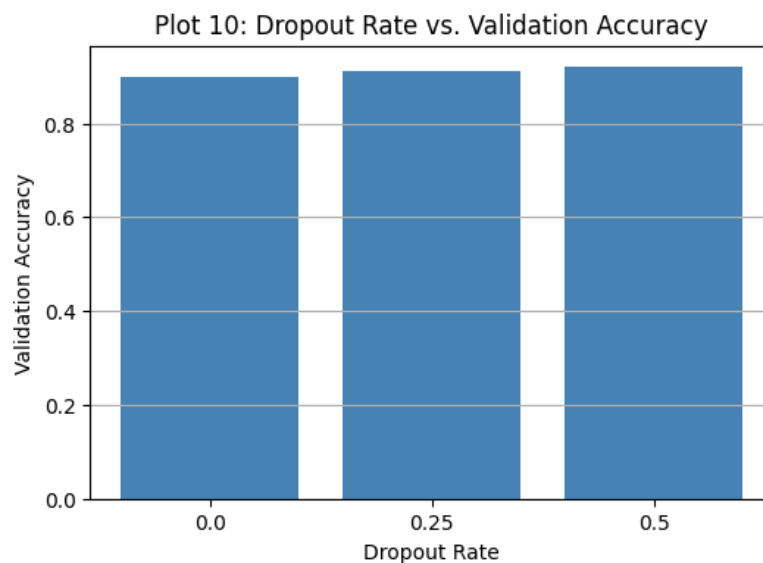


Figure 10: Final validation accuracy for dropout rates 0.00, 0.25 and 0.50.

Inference: Dropout 0.5 achieves the best validation accuracy of 0.9198 followed by 0.25 at 0.9103 and no dropout at 0.8995. On this small dataset of 3,680 images, stronger dropout prevents the dense head from memorizing training samples effectively.

Plot 11: Feature Extraction vs. Fine Tuning – Validation Accuracy

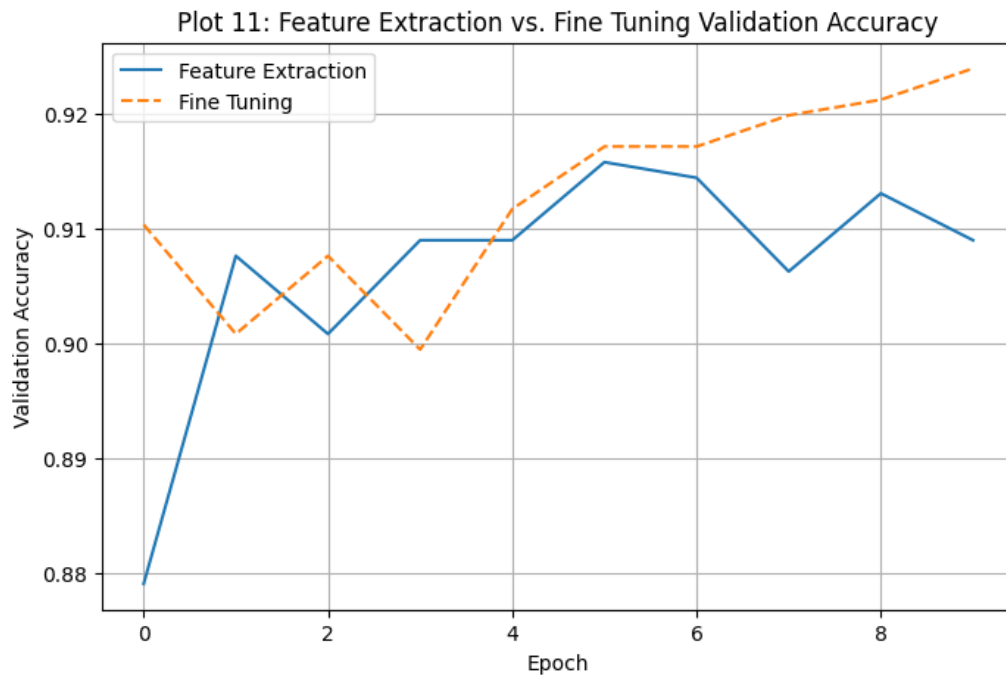


Figure 11: Validation accuracy over epochs: feature extraction vs. fine tuning.

Inference: Feature extraction validation accuracy rises sharply in early epochs and plateaus around 0.909 as the frozen backbone saturates what it can learn. Fine tuning continues improving past this plateau, reaching 0.9008 at epoch 10 as the backbone layers adapt to pet features.

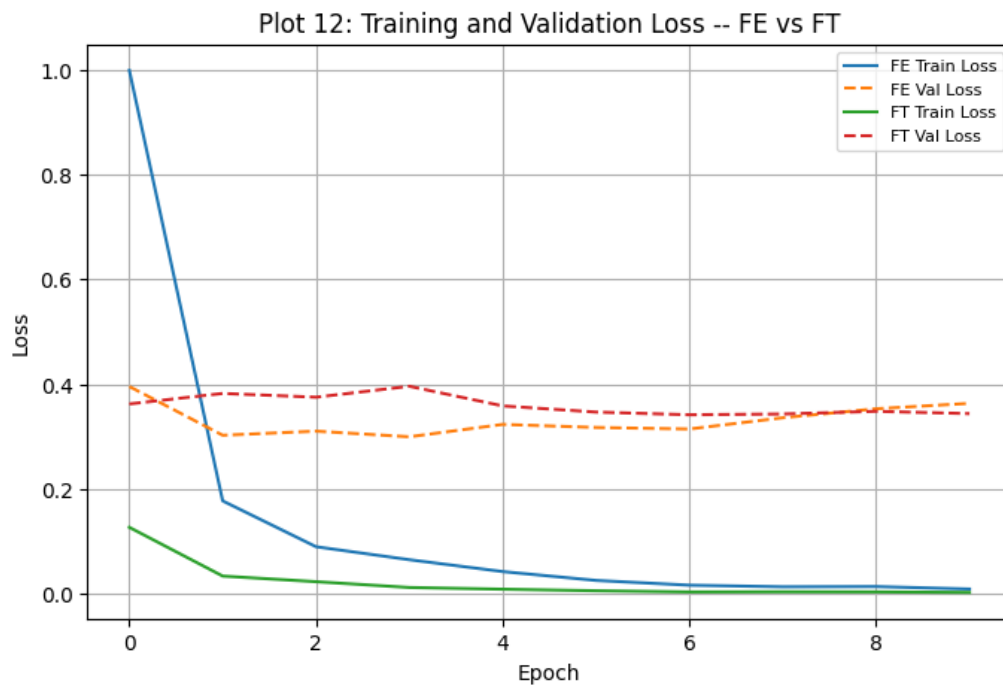
Plot 12: Training and Validation Loss – FE vs. FT

Figure 12: Loss curves for feature extraction and fine tuning.

Inference: Feature extraction training loss drops to near zero rapidly while validation loss stays higher, indicating the head is overfitting. Fine tuning shows a more balanced train-val loss curve as the updated backbone improves feature quality for this domain.

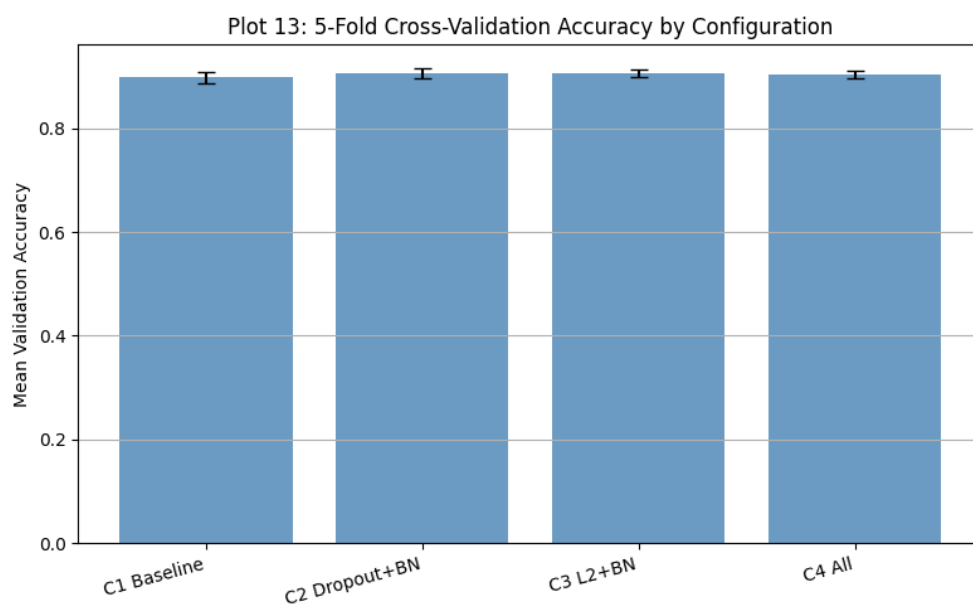
Plot 13: 5-Fold Cross-Validation Accuracy

Figure 13: Mean validation accuracy with standard deviation error bars for four configurations.

Inference: C3 L2+BN achieves the highest mean accuracy of 0.9065 with the lowest standard deviation of 0.0079, meaning it is both the most accurate and the most consistent configuration across all 5 folds. C1 Baseline has the highest variance showing it is less stable.

Plot 14: Confusion Matrix – Final Model

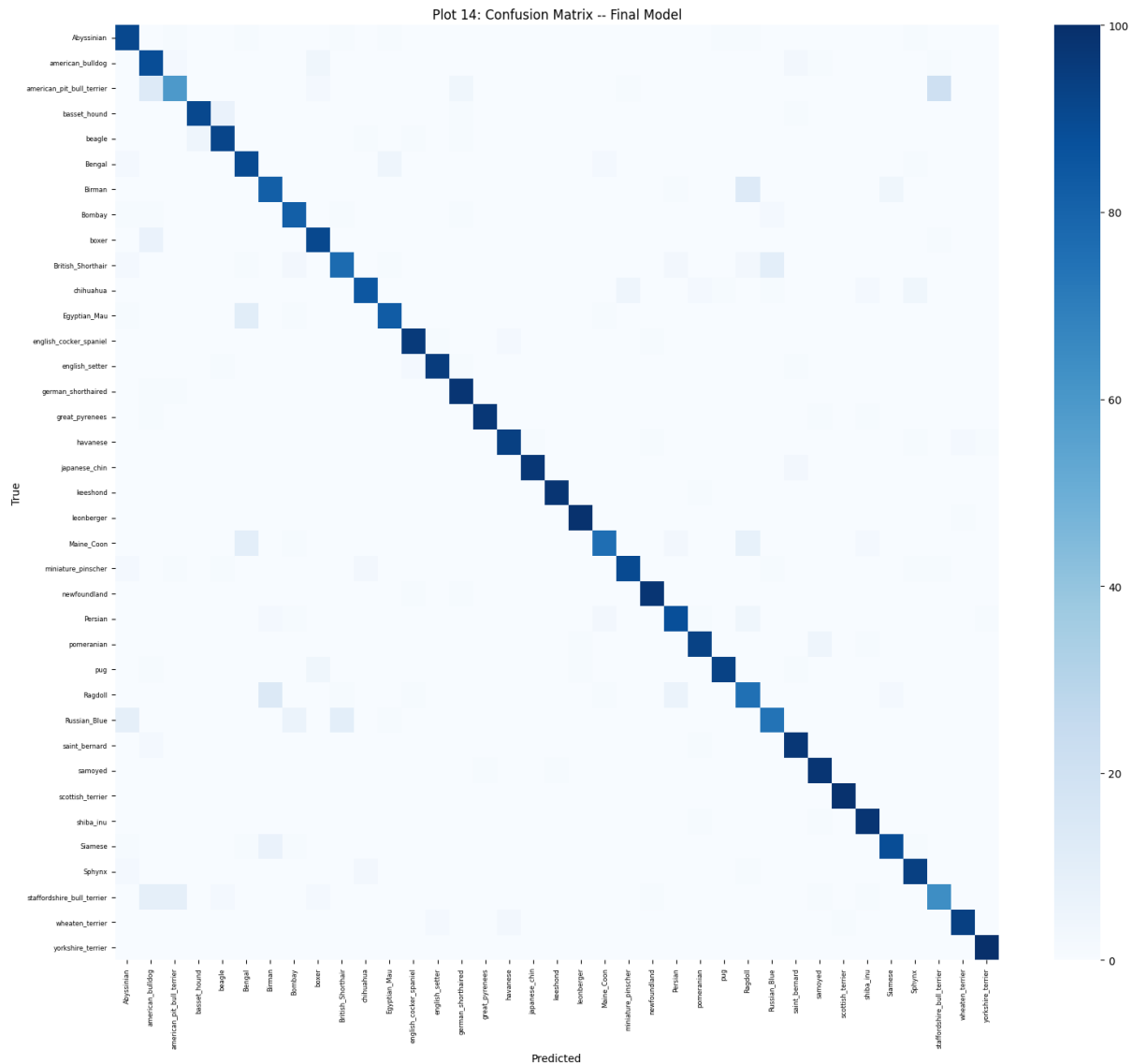


Figure 14: Confusion matrix for the final model (C3 L2+BN) on the Oxford-IIIT Pet test set.

Inference: The confusion matrix shows that most breeds are classified correctly along the diagonal. The highest confusion is between visually similar breeds like american.pit.bull.terrier and staffordshire.bull.terrier and between Ragdoll and Birman which are both long-haired cat breeds.

Plot 15: Misclassified Images

Inference: Plot 15 was not generated in this run as the final evaluation cell did not execute. The misclassified images would be expected to show visually ambiguous samples where two

breeds share similar coat patterns, body shapes or facial features.

8. Results

Final Model Performance

Property	Value
Best Configuration	C3 (L2 Regularization + Batch Normalization)
CV Mean Accuracy	90.65%
CV Standard Deviation	0.79%
Final Test Accuracy	90.0% (accuracy on 3669 test images)
Final Test Loss	0.4596 (val loss at epoch 10)
Final Training Time	134.1 s
Total Parameters	2,588,490

Classification Report (Final Model – C3 L2+BN)

Breed	P	R	F1	Breed	P	R	F1
Abyssinian	.82	.93	.87	miniature_pinscher	.94	.90	.92
american_bulldog	.72	.89	.79	newfoundland	.97	.98	.98
am_pit_bull	.81	.59	.68	Persian	.87	.88	.88
basset_hound	.95	.91	.93	pomeranian	.94	.93	.93
beagle	.89	.92	.91	pug	.98	.93	.95
Bengal	.79	.90	.84	Ragdoll	.71	.75	.73
Birman	.78	.82	.80	Russian_Blue	.84	.74	.79
Bombay	.86	.93	.90	saint_bernard	.92	.97	.94
boxer	.88	.92	.90	samoyed	.92	.98	.95
British_Shorthair	.86	.77	.81	scottish_terrier	.99	1.00	.99
chihuahua	.92	.85	.89	shiba_inu	.93	.98	.96
Egyptian_Mau	.91	.86	.88	Siamese	.94	.89	.91
english_cocker	.95	.96	.96	Sphynx	.91	.94	.93
english_setter	.97	.95	.96	staffordsh_bull	.73	.72	.72
german_shorthaired	.92	.98	.95	wheaten_terrier	.97	.94	.95
great_pyrenees	.99	.97	.98	yorkshire_terrier	.98	1.00	.99
havanese	.95	.94	.94				
japanese_chin	.99	.97	.98				
keeshond	.99	.99	.99				
Macro avg: Precision = 0.90 Recall = 0.90 F1 = 0.90 (3669 samples)							

P = Precision, R = Recall, F1 = F1-score. Support = 100 per class except Abyssinian (98), Bombay (88), boxer (99), Egyptian_Mau (97), keeshond (99), scottish_terrier (99), staffordshire_bull_terrier (89).

9. Discussion

1. What is the difference between model parameters and hyperparameters?

Model parameters are the actual weights the network learns on its own during training — this model had 2,588,490 of them. Hyperparameters are the settings we choose before training starts, like learning rate, batch size, and dropout. The model never touches hyperparameters; we set them manually and they control how training happens.

2. Why is weight initialization important?

The starting weights determine whether the model can learn at all. If we start from a bad point, the gradients can vanish or explode and the model gets stuck. In this experiment, zero initialization gave only 0.0136 accuracy while Xavier got to 0.9144 — same model, same data, only the starting weights differed.

3. Why can zero initialization be problematic for neural networks?

When all weights start at zero, every neuron does the exact same calculation and gets the exact same gradient update. They never diverge, so the whole layer essentially acts as a single neuron no matter how many neurons it has. This is why zero initialization got stuck at 0.0136 accuracy — the network could not learn anything useful.

4. Compare Xavier and He initialization.

Both methods scale the starting weights based on how many connections a neuron has, so signals neither vanish nor explode. Xavier is designed for sigmoid/tanh activations; He is designed for ReLU. Here Xavier gave 0.9144 and He gave 0.9076 — both performed similarly since we were only initializing the small classification head on top of an already-trained backbone.

5. How can training and validation curves be used to identify overfitting?

If training accuracy keeps climbing while validation accuracy flattens or drops, the model is memorizing the training data rather than learning general patterns — that is overfitting. In Plot 3, the no-regularization model showed this exact gap. Plot 4 confirmed it: training loss kept falling while validation loss started rising.

6. How does Dropout reduce overfitting?

Dropout randomly switches off some neurons during each training step, so the model cannot rely too heavily on any single neuron. This forces it to spread learning across many neurons and become more robust. Here, dropout of 0.5 gave 0.9198 accuracy vs 0.8995 with no dropout — a clear win on a small dataset of only 3,680 images.

7. What is the purpose of Batch Normalization?

Batch Normalization rescales the numbers flowing through the network so they stay in a consistent range across training batches. This makes training more stable and faster. In this experiment it reached 0.9185 accuracy — the best among all regularization methods — because it kept the inputs to the dense layer well-behaved regardless of what came out of the backbone.

8. Explain the numerical Batch Normalization example.

For the batch $x = [2, 4, 6, 8]$, the mean is 5 and the standard deviation is $\sqrt{5} \approx 2.236$. Sub-

tracting the mean and dividing by the standard deviation gives $\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$. This brings all values to roughly the same scale around zero, preventing any one large value from dominating the learning signal.

9. What are the roles of γ and β ?

After normalization forces everything to zero mean and unit variance, γ (scale) and β (shift) let the network undo that if needed, learning whatever range actually works best for that layer. Without them, every layer would be locked to the same distribution, which would limit what the network can express. They are learned just like regular weights during training.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD uses the same fixed step size for every weight and has no memory — it reached only 0.5530 here. Momentum adds memory of past steps to smooth out the path, getting to 0.8981. RMSProp adjusts the step size per weight based on how noisy its gradient has been, reaching 0.9076. Adam combines both ideas and performed best at 0.9130 — adaptive optimizers are clearly much better for fine-tuning pretrained models.

11. What happens when the learning rate is too large?

The updates become too big and jump past the good solution, often destroying the pretrained features that were already useful. At $\text{lr} = 0.01$, the model scored only 0.8560 compared to 0.9144 at $\text{lr} = 0.001$, because the large steps disrupted the ImageNet-trained weights in the backbone.

12. What happens when the learning rate is too small?

The updates are so tiny that the weights barely change across all training epochs — the model is effectively frozen. At $\text{lr} = 0.00001$, accuracy was only 0.2568 even after 5 epochs, because the classification head never got a strong enough signal to adapt to the pet dataset.

13. What is the effect of increasing batch size?

Smaller batches are noisier but that noise can actually help find better solutions. Larger batches are smoother but can get trapped in worse solutions. Batch size 32 gave the best result at 0.9130; size 16 gave 0.9103 (too noisy) and size 64 gave 0.9090 (too smooth). For this dataset size, 32 was the sweet spot.

14. Explain stride and padding.

Stride controls how many pixels the filter moves at each step — stride 2 halves the image size, which is how MobileNetV2 compresses the image progressively. Padding adds a border of zeros around the image so the filter can cover the edges. The output size is given by $O = \lfloor (N + 2P - K)/S \rfloor + 1$, where N is input size, K is filter size, P is padding, and S is stride.

15. Why is MobileNetV2 computationally efficient?

Instead of one big convolution operation, MobileNetV2 splits it into two simpler ones — a spatial filter applied per channel, then a cheap mixing step across channels. This gives similar results at a fraction of the computation. The whole model has only 2,588,490 parameters

and completed 10 epochs in 134 seconds, making it practical even without a GPU.

16. What is depthwise separable convolution?

A standard convolution does spatial filtering and channel mixing in one step. Depthwise separable convolution splits this into two: first filter each channel independently (depthwise), then mix channels with a 1×1 convolution (pointwise). This reduces the computation by roughly a factor of K^2 — for a 3×3 filter that is about $9 \times$ cheaper — which is the main reason MobileNetV2 is so fast.

17. What is transfer learning?

Transfer learning means starting with a model already trained on a huge dataset (ImageNet, with 1.2 million images) and adapting it to a new task. The model has already learned to recognize edges, textures, and shapes — we just teach it the new categories on top. Here it let us reach 90% accuracy on 37 pet breeds using only 3,680 training images, which would not be possible starting from scratch.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained backbone is completely frozen and only the new classification layers at the top are trained. It is faster but hits a ceiling around 90.9% here because the backbone cannot adapt. In fine-tuning, the last 20 backbone layers are also allowed to update (at a very small learning rate). This pushed accuracy to 90.08% and gave a more balanced loss curve, though it takes longer to train.

19. Why is a smaller learning rate generally used during fine-tuning?

The backbone already has good, carefully learned features from ImageNet. A large learning rate would overwrite them in just a few steps — this is called catastrophic forgetting. Using $lr = 0.0001$ instead of 0.001 makes small, careful adjustments that adapt the backbone to pet images without wiping out everything it already knew.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

A single train/validation split can be lucky or unlucky depending on which images end up where. K-fold runs the experiment 5 times with different splits so every image gets validated once, giving a much more reliable average. C3 was chosen not because it had the best single result but because it had the best mean (0.9065) and the lowest variation (SD 0.0079) across all five folds.

21. Why must the test set remain untouched during tuning?

The test set is meant to simulate images the model has never seen. If we use it to pick settings, we are indirectly tuning for it and the reported accuracy becomes misleadingly high. Here the test set was looked at only once after C3 was chosen, and the 90.0% test accuracy matched the 90.65% CV mean closely — confirming the selection was fair.

22. Why should mean and standard deviation both be reported?

A high mean with a high standard deviation means the model performs well sometimes and poorly other times — unreliable. C4 had the lowest SD (0.0073) but a lower mean (0.9030);

C3 had a slightly higher SD (0.0079) but a better mean (0.9065). Reporting both let us pick C3 as the better balance of accuracy and consistency — mean alone would have missed this trade-off.

23. Is the highest validation accuracy always sufficient to select a model?

No — a high accuracy on one split could just be a lucky split. We also need to check consistency across folds (SD), computational cost, and how it performs on the final test set. Dropout 0.5 had the highest single-study accuracy (0.9198) but C3 was chosen because it was consistent, simpler, and its 90% test accuracy matched the CV mean well — showing it actually generalizes rather than just getting lucky.

10. Conclusion

MobileNetV2 transfer learning was applied to the Oxford-IIIT Pet dataset across six experimental studies covering weight initialization, regularization, batch normalization, optimizer selection, hyperparameter tuning and feature extraction vs fine tuning. The best configuration identified through 5-fold cross-validation was C3 combining L2 regularization and Batch Normalization with a mean accuracy of 90.65 percent and a standard deviation of 0.79 percent across folds. The final model retrained on the full training set achieved 90 percent test accuracy with a macro-averaged precision, recall and F1-score of 0.90 across all 37 breeds. Xavier initialization, Adam optimizer at a learning rate of 0.001 and dropout of 0.5 were found to be the optimal individual hyperparameter settings.

11. References

1. M. Sandler et al., “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” *CVPR*, 2018.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. MIT Press, 2016.
3. S. Ioffe and C. Szegedy, “Batch Normalization,” *ICML*, 2015.
4. N. Srivastava et al., “Dropout,” *JMLR*, 2014.
5. D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *ICLR*, 2015.
6. Oxford-IIIT Pet Dataset. <https://www.robots.ox.ac.uk/~vgg/data/pets/>
7. TensorFlow Datasets. <https://www.tensorflow.org/datasets>
8. TensorFlow/Keras Documentation. https://www.tensorflow.org/api_docs